

Functional Foundations

A Mathematical & Practical Guide to Functional Programming and
Category Theory

Kanshu Yokoo

March 8, 2026

Preface

This book is for anyone eager to study functional programming, especially those who wish to explore Category Theory through that lens. I have written this for two people: my younger self as a high school student, and the person I am today. Of course, it is also for anyone who shares those same interests. When I was younger, the opportunity to learn Category Theory alongside functional programming simply didn't exist.

This textbook bridges the gap between formal mathematics and practical software engineering using Python and Haskell. It is designed specifically for students with a high school mathematical background.

Contents

1	Introduction to Mathematical Functions	1
1.1	What is a Function?	1
1.1.1	Domains and Codomains	1
1.2	From Mathematics to Code	1
1.2.1	A Simple Mapping	2
1.3	An Early Glimpse at Category Theory	2
2	The Calculus of Functions	5
2.1	Introduction to Lambda Calculus	5
2.1.1	Anonymous Functions in Code	5
2.2	Bound vs. Free Variables	6
2.3	The Rules of Computation	6
2.3.1	Alpha-Conversion (α -conversion)	6
2.3.2	Beta-Reduction (β -reduction)	7
2.4	Why This Matters	7
3	The Functional Paradigm	9
3.1	Two Ways to Think About Programs	9
3.1.1	The Imperative Approach: How to Do It	9
3.1.2	The Declarative Approach: What to Compute	10
3.2	Pure Functions	10
3.2.1	Pure vs. Impure in Python	10
3.3	Side Effects	11
3.4	Referential Transparency	11

Chapter 1

Introduction to Mathematical Functions

1.1 What is a Function?

In mathematics, a function is a fundamental concept that describes a reliable relationship between two sets of data. When we say a function “ f ” takes an input “ x ” and produces an output “ y ”, we write this relationship as:

$$f(x) = y$$

You can think of a mathematical function as a well-behaved machine. If you drop a specific wooden block (the input) into the top of the machine, it will *always* paint it red (the output) and drop it out the bottom. It never paints it blue one day and red the next; the result is strictly determined by the input.

1.1.1 Domains and Codomains

To properly use our “machine,” we need to know exactly what kind of inputs it accepts and what kind of outputs it promises to produce. In formal mathematics, we define these boundaries using **Sets**.

- **Domain:** The set of all possible *valid inputs*. If our machine only accepts square wooden blocks, the Domain is the set of all square wooden blocks.
- **Codomain:** The set of all *possible outputs* the function could mathematically produce. If our machine paints blocks red, the Codomain is the set of all red objects.

When defining a function formally, mathematicians write its signature like this:

$$f : A \rightarrow B$$

This is read as “ f is a function from set A to set B ”. Set A is the domain, and set B is the codomain. Notice how this resembles an arrow pointing from the source to the destination.

1.2 From Mathematics to Code

In standard imperative programming (like writing typical Python scripts), a “function/method” is often just a sequence of instructions grouped together. It might take an input, look at a global variable, print something to the screen, change a database, and then return a value.

That is **not** a true mathematical function. A true mathematical function (often called a “pure function” in programming) has two strict rules:

1. It must always produce the exact same output for the same input.
2. It must not cause any observable side effects (no printing, no modifying global variables).

Let us look at how mathematics translates to code using Python and Haskell.

1.2.1 A Simple Mapping

Consider a mathematical function that squares an integer:

$$f : \mathbb{Z} \rightarrow \mathbb{Z}$$

$$f(x) = x^2$$

Here is how we represent that pure mathematical function in Python:

```

1 def square(x: int) -> int:
2     """
3     A pure function mapping from the set of Integers to the set of Integers.
4     """
5     return x * x

```

Listing 1.1: Python: A pure squaring function

Notice the use of type hints (`int`). In functional programming, types are the equivalent of mathematical sets. The type hint `int -> int` is the exact programmatic equivalent of the mathematical signature $\mathbb{Z} \rightarrow \mathbb{Z}$.

Now, let us look at Haskell, a pure functional language. In Haskell, you are *forced* to write pure mathematical functions.

```

1 -- The type signature is the mathematical definition:
2 -- f is a function from the set of Int to the set of Int
3 square :: Int -> Int
4
5 -- The implementation is the mapping:
6 square x = x * x

```

Listing 1.2: Haskell: A pure squaring function

In Haskell, the signature `square :: Int -> Int` is front and center. It explicitly declares that `square` is a mapping from the set of Integers to the set of Integers.

1.3 An Early Glimpse at Category Theory

You have now seen three concepts:

- Sets of data (like Integers or Strings).
- Functions that map data from one set to another.
- The idea that these mappings behave like arrows ($A \rightarrow B$).

Without realizing it, you have just stepped into **Category Theory**.

Category Theory is an abstract branch of mathematics that studies structures and systems of structures. Instead of looking at the specific data *inside* a set (like the number 4 inside the set of integers), Category Theory zooms out and looks at how entire sets relate to each other through functions.

A **Category** is simply a collection of two things:

1. **Objects:** These are the “things” in your category. In programming, you can think of Objects as your **Types** (like `int`, `str`, or `List`). In mathematics, these are often Sets.

2. **Morphisms (Arrows):** These are the connections between the Objects. In programming, these are your **Functions**. If you have a function that takes an `int` and returns a `str`, that function is a morphism pointing from the `int` object to the `str` object.

When you write a pure function in Python or Haskell, you are defining a morphism between two objects in the “Category of Types” (often called **Hask** in the Haskell world).

As we progress through this book, we will rely heavily on this simple visual metaphor. By treating types as dots (objects) and functions as arrows (morphisms) connecting those dots, complex programming concepts will become elegantly simple.

Chapter 2

The Calculus of Functions

2.1 Introduction to Lambda Calculus

In the 1930s, mathematician Alonzo Church invented a formal system called **Lambda Calculus** (λ -calculus) to investigate the foundations of mathematics. While Alan Turing was inventing the Turing Machine (the theoretical basis for imperative programming languages like C, Java, and Python’s procedural aspects), Church was inventing Lambda Calculus, which became the theoretical foundation for all functional programming languages.

Lambda Calculus is astonishingly simple. It consists of only three things:

1. **Variables:** Symbols like x, y, z .
2. **Abstractions (Function Definitions):** Creating a function, denoted by the Greek letter Lambda (λ).
3. **Applications (Function Calls):** Applying a function to an argument.

Let us look at a simple mathematical function: $f(x) = x + 1$. In Lambda Calculus, we drop the name “ f ” entirely and define just the mapping. We write it using a lambda abstraction:

$$\lambda x.x + 1$$

This reads as: “A function that takes a variable x (the part before the dot) and returns $x + 1$ (the body after the dot).”

2.1.1 Anonymous Functions in Code

Because these functions don’t have a name like “ f ”, they are called **Anonymous Functions**. Modern programming languages have adopted this concept directly.

Here is how you write anonymous functions in Python:

```
1 from typing import Callable
2
3 # A standard named function
4 def add_one_named(x: int) -> int:
5     return x + 1
6
7 # The Lambda Calculus equivalent: an anonymous function
8 # Syntax: lambda bound_variable: body
9 add_one_lambda: Callable[[int], int] = lambda x: x + 1
10
11 # Beta-reduction in Python (Application)
12 # (lambda x: x + 1)(5)
13 result = (lambda x: x + 1)(5)
```

```
14 print(result) # Output: 6
```

Listing 2.1: Python: Lambda (Anonymous) Functions

And here is the exact same concept in Haskell, which uses the backslash `\` as a typographical substitute for the Greek letter λ :

```
1 -- A standard named function
2 addOneNamed :: Int -> Int
3 addOneNamed x = x + 1
4
5 -- The Lambda Calculus equivalent: an anonymous function
6 -- Syntax: \bound_variable -> body
7 -- (The backslash '\' looks like a lambda 'lambda')
8 addOneLambda :: Int -> Int
9 addOneLambda = \x -> x + 1
10
11 -- Beta-reduction in Haskell (Application)
12 -- (\x -> x + 1) 5
13 result :: Int
14 result = (\x -> x + 1) 5
15 -- result evaluates to 6
```

Listing 2.2: Haskell: Anonymous Functions

2.2 Bound vs. Free Variables

To understand how Lambda Calculus evaluates, we must distinguish between bound and free variables.

Look at this expression:

$$\lambda x. x + y$$

- The variable x is **bound**. It is declared in the “head” (λx) of the function. Whoever calls the function will provide the value for x .
- The variable y is **free**. It is not defined in the head. For this function to make sense, y must be defined somewhere else in the surrounding environment.

A lambda expression where every variable is bound is called a **Combinator**. The simplest combinator is the Identity Function ($\lambda x. x$), which just returns whatever is passed to it.

2.3 The Rules of Computation

Turing Machines compute by moving tape and changing states. Lambda Calculus computes using just two simple text-replacement rules: Alpha-conversion and Beta-reduction.

2.3.1 Alpha-Conversion (α -conversion)

Alpha-conversion is the rule of **renaming**. It states that the specific name of a bound variable does not matter.

$$\lambda x. x \text{ is exactly the same as } \lambda y. y$$

In Python, `lambda x: x` and `lambda y: y` are identical functions. Changing the variable name does not change what the function *does*. This allows us to avoid naming collisions when applying functions to other functions.

2.3.2 Beta-Reduction (β -reduction)

Beta-reduction is the rule of **application**. It is the actual process of “running” or “calling” the function. It works by substituting the input argument into the body of the function wherever the bound variable appears.

Consider applying our $+1$ function to the number 5:

$$(\lambda x.x + 1) 5$$

To perform a Beta-reduction, we take the input (5), replace every instance of the bound variable (x) in the body with it, and drop the lambda head:

$$(\lambda x.x + 1) 5 \xrightarrow{\beta} 5 + 1 \rightarrow 6$$

If you string multiple lambda applications together, Beta-reduction is how you “execute” the program, step-by-step, until you arrive at the final answer.

2.4 Why This Matters

In functional programming, whether you are using Python’s `map()` or building complex Haskell architectures, your program is fundamentally a massive expression of nested functions. The compiler or interpreter executes your program by performing Beta-reductions until no more functions can be applied.

Understanding Lambda Calculus gives you the mental model to see through the syntax of a language and understand the pure dataflow underneath.

Chapter 3

The Functional Paradigm

3.1 Two Ways to Think About Programs

There are two fundamentally different philosophies for writing programs. Understanding the difference between them is the cornerstone of functional programming.

3.1.1 The Imperative Approach: How to Do It

Imperative programming is the most common style, used in languages like C, Java, and standard Python. You give the computer a sequence of commands—a step-by-step recipe that describes *how* to achieve a result by changing the state of the program.

Consider a program that calculates the sum of a list of numbers using the imperative style:

```
1 from typing import List
2
3 # --- IMPERATIVE STYLE ---
4 # How to compute: step-by-step mutation of state.
5 def imperative_sum(numbers: List[int]) -> int:
6     total: int = 0
7     for n in numbers:
8         total = total + n # mutate the variable at each step
9     return total
10
11 # --- DECLARATIVE / FUNCTIONAL STYLE ---
12 # What the result is: expressed as a recursive relationship.
13 def functional_sum(numbers: List[int]) -> int:
14     if not numbers:
15         return 0
16     return numbers[0] + functional_sum(numbers[1:])
17
18 # Alternatively, using Python's built-in pure function:
19 from functools import reduce
20 declarative_sum = lambda numbers: reduce(lambda acc, x: acc + x, numbers, 0)
21
22 print(imperative_sum([1, 2, 3, 4, 5])) # 15
23 print(functional_sum([1, 2, 3, 4, 5])) # 15
24 print(declarative_sum([1, 2, 3, 4, 5])) # 15
```

Listing 3.1: Python: Imperative style summation

In this style, we use a variable `total` that we *mutate* (change) at each step. The program is a sequence of state changes: first `total` is 0, then 1, then 3, then 6, and so on.

3.1.2 The Declarative Approach: What to Compute

Declarative programming is the style that emerges from Lambda Calculus. Instead of describing *how* to compute a result, you describe *what* the result **is**—as a mathematical expression. You do not give step-by-step instructions; you express relationships between values.

Haskell is a declarative language at its core. Look at its equivalent:

```

1 -- Haskell is declarative by nature.
2 -- There are no for-loops or mutable variables.
3
4 -- DECLARATIVE STYLE: Recursive definition of sum.
5 -- This declares "what" a sum IS, not "how" to compute it.
6 mySum :: [Int] -> Int
7 mySum [] = 0 -- base case: the sum of an empty list is 0
8 mySum (x:xs) = x + mySum xs -- recursive case: head + sum of the tail
9
10 -- Haskell's built-in 'sum' is defined the same way via 'foldr'.
11 -- sum [1,2,3,4,5] == 15

```

Listing 3.2: Haskell: Declarative style summation

Notice that in Haskell's `mySum`, there is no loop, no counter variable, and no mutation. The function *declares* what a sum is—a recursive relationship—and the runtime figures out the rest.

The key principle is: **in functional programming, you compose expressions, not statements.**

3.2 Pure Functions

At the heart of the functional paradigm is the concept of a **Pure Function**—a function that behaves exactly like a mathematical function, following two rules:

1. **Deterministic:** Given the same input, it always returns the exact same output. No randomness, no time-dependence, no reading from a database.
2. **No Side Effects:** It does not change anything observable outside its own scope. It does not print to the screen, modify a global variable, write to a file, or send a network request.

In Category Theory terms, a pure function is a genuine *morphism*—a reliable arrow from one type (object) to another. If the function could secretly modify the world around it, it would no longer be a true arrow between types.

3.2.1 Pure vs. Impure in Python

Python allows both pure and impure functions. It is the programmer's discipline that enforces purity. Here is a clear contrast:

A **pure function**—its only output is its return value, entirely determined by its input:

```

def add(x: int, y: int) -> int:
    return x + y

```

An **impure function**—it reads from and modifies the external world:

```

count: int = 0

def increment_and_add(x: int) -> int:
    global count
    count += 1 # side effect: modifies external state
    print(count) # side effect: performs I/O
    return x + count # result depends on hidden external state

```


The impure version is unpredictable. Its output for `increment_and_add(5)` changes every time it is called—even with the same argument. This makes it extremely difficult to reason about, test, and compose with other functions.

3.3 Side Effects

A **side effect** is any observable interaction a function has with the outside world beyond returning a value. Common side effects include:

- Modifying a global or mutable variable.
- Writing to the console (`print()`).
- Reading from or writing to a file or database.
- Making a network request.
- Raising or catching an exception in a way that alters control flow.

Side effects are not inherently “evil”—a program that never prints anything, writes to a file, or talks to a database is not very useful. The functional approach is not to *eliminate* side effects, but to **isolate and control** them, keeping the majority of the program logic in pure, predictable functions. The impure parts (I/O, mutations) are pushed to the outer edges of the system.

Haskell enforces this separation at the type level using the `IO` type. Any function that performs a side effect *must* declare this in its type signature (e.g., `printSquare :: Int -> IO ()`), making impurity explicit and impossible to accidentally introduce into pure code.

3.4 Referential Transparency

Referential Transparency is a formal property that arises directly from using pure functions. An expression is referentially transparent if it can be replaced with its computed value without changing the behaviour of the program.

Consider this expression in Python:

```
result = add(2, 3) + add(2, 3)
```

Because `add` is a pure function that always returns 5 for inputs (2, 3), we can freely substitute:

```
result = 5 + 5
```

The program behaves identically. `add(2, 3)` is referentially transparent because it is pure.

Now consider `increment_and_add(5) + increment_and_add(5)`. You *cannot* substitute `increment_and_add(5)` with its first computed value, because the second call will produce a different result (it increments `count` again). This violates referential transparency.

Referential transparency has a profound practical consequence: it enables the compiler (and the programmer) to reason about code through **equational reasoning**—replacing expressions with equal expressions, just like in algebra. This is the mathematical bedrock of the functional paradigm, and it is why functional code is generally easier to test, refactor, and compose.

In category theoretic terms, referential transparency is what lets us treat functions as genuine morphisms—arrows that reliably map objects to objects, without hidden context.

Chapter 4

Functions as First-Class Citizens

4.1 What Does “First-Class” Mean?

In many traditional programming languages, functions and data are kept strictly separate. You can store an integer in a variable, pass a string to a function, or return a boolean. But you cannot do those things *with a function itself*. The function is a static piece of architecture, while the data flows through it.

In functional programming, **functions are data**.

When we say a language treats functions as “First-Class Citizens,” we mean that a function has the exact same rights and privileges as any other piece of data (like an integer or a string). Specifically, you can:

1. Assign a function to a variable.
2. Pass a function as an argument to another function.
3. Return a function as the result of another function.

4.2 Passing Functions as Data

Let us look at how this works in practice. Suppose we want to apply a transformation to a number. Instead of hardcoding the transformation, we can pass the transformation *as a function*.

```
1 from typing import Callable, List
2
3 # A simple pure function
4 def add_one(x: int) -> int:
5     return x + 1
6
7 def multiply_by_two(x: int) -> int:
8     return x * 2
9
10 # HIGHER-ORDER FUNCTION
11 # Takes a function 'f' as an argument.
12 def apply_twice(f: Callable[[int], int], x: int) -> int:
13     return f(f(x))
14
15 # We can pass different functions like data
16 print(apply_twice(add_one, 5))          # (5 + 1) + 1 = 7
17 print(apply_twice(multiply_by_two, 5)) # (5 * 2) * 2 = 20
18
19 # MAP, FILTER, REDUCE in Python
20 numbers: List[int] = [1, 2, 3, 4, 5]
21
22 # map applies a function to every item
```

```

23 squares = list(map(lambda x: x * x, numbers))
24 print(squares) # [1, 4, 9, 16, 25]
25
26 # filter keeps items where the function returns True
27 evens = list(filter(lambda x: x % 2 == 0, numbers))
28 print(evens) # [2, 4]

```

Listing 4.1: Python: Passing a function as an argument

In the `apply_twice` function, the parameter `f` is not a number or a string; it is a function. We can swap out `f` for any function we want, making `apply_twice` incredibly flexible.

4.3 Higher-Order Functions

A function that does at least one of the following is called a **Higher-Order Function** (HOF):

- Takes one or more functions as arguments.
- Returns a function as its result.

Calculus students might recognize this concept. The derivative operator $\frac{d}{dx}$ is a higher-order function: it takes a function as an input (like $f(x) = x^2$) and returns a new function as an output ($f'(x) = 2x$).

4.3.1 The Holy Trinity: Map, Filter, and Reduce

Because functional programmers use Higher-Order Functions constantly, three specific HOFs have become the universal standard for processing lists of data: `map`, `filter`, and `reduce` (often called `fold`).

Map

`map` takes a function and a list, and applies that function to every single item in the list, returning a new list.

If you have a list of integers `[1, 2, 3]` and you `map` a square function over it, you get `[1, 4, 9]`. You never write a `for`-loop; you just declare the transformation.

```

1  -- A simple pure function
2  addOne :: Int -> Int
3  addOne x = x + 1
4
5  -- HIGHER-ORDER FUNCTION
6  -- Takes a function (Int -> Int) and an Int, returns an Int
7  applyTwice :: (Int -> Int) -> Int -> Int
8  applyTwice f x = f (f x)
9
10 -- applyTwice addOne 5    == 7
11 -- applyTwice (*2) 5     == 20
12
13 -- MAP AND FILTER in Haskell
14
15 numbers :: [Int]
16 numbers = [1, 2, 3, 4, 5]
17
18 -- Map applies a function to an entire list
19 squares :: [Int]
20 squares = map (\x -> x * x) numbers
21 -- squares == [1, 4, 9, 16, 25]
22
23 -- Filter keeps items matching the predicate

```

```
24 evens :: [Int]
25 evens = filter (\x -> x `mod` 2 == 0) numbers
26 -- evens == [2, 4]
27
28 -- Fold (Reduce) combines the list elements
29 totalSum :: Int
30 totalSum = foldl (+) 0 numbers
31 -- totalSum == 15
```

Listing 4.2: Haskell: Map and Filter

Filter

`filter` takes a *predicate* function (a function that returns True or False) and a list. It returns a new list containing only the items for which the predicate returned True.

If you `filter` the list `[1, 2, 3, 4]` with an `is_even` function, you get `[2, 4]`.

Reduce (Fold)

`reduce` (or `fold` in Haskell) takes a function, an initial starting value, and a list. It uses the function to combine the elements one by one until the list is reduced to a single value.

For example, reducing the list `[1, 2, 3]` with an addition function and a starting value of 0 will compute $((0 + 1) + 2) + 3 = 6$.

4.4 A Gentle Introduction to Functors

In Category Theory, we established that a Category consists of Objects (types) and Morphisms (functions).

What happens if we want to map relationships *between entire categories*?

In Category Theory, a mapping between categories is called a **Functor**. In programming, you can think of a Functor as any container or context (like a List) that can be mapped over.

When you use the `map` function on a List of Integers to turn it into a List of Strings, you are using the List Functor. The Functor reaches inside the container, applies your function to the data, and wraps the results back up in a new container, all while preserving the structure of the container itself.

Functions as first-class citizens are what make concepts like Functors possible in code. By passing a function into `map`, you are lifting that simple data transformation into a higher mathematical dimension.

